



Redes Neuronales

Ing. Jaime Antonio Huaytalla Pariona
jhuaytalla@uncp.edu.pe

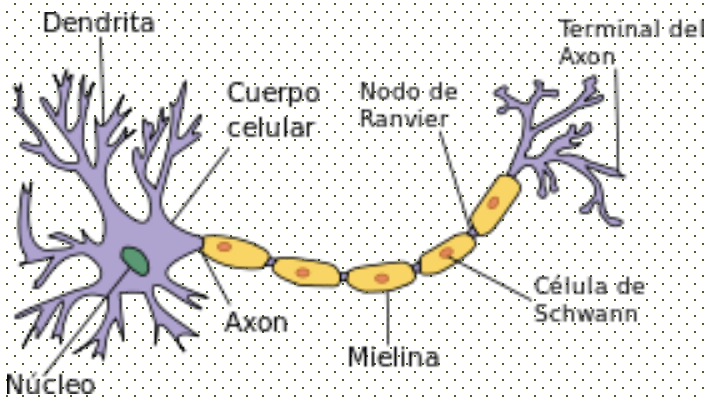
Temas

1. **Introducción y Conceptos Básicos**
2. Redes Neuronales Simples
3. Backpropagation
4. Redes Neuronales Profundas

Redes Neuronales

- También llamadas “Redes Neuronales Artificiales” (ANN: *Artificial Neural Networks*)

- Se inspiran en:
  - Neuronas del cerebro humano

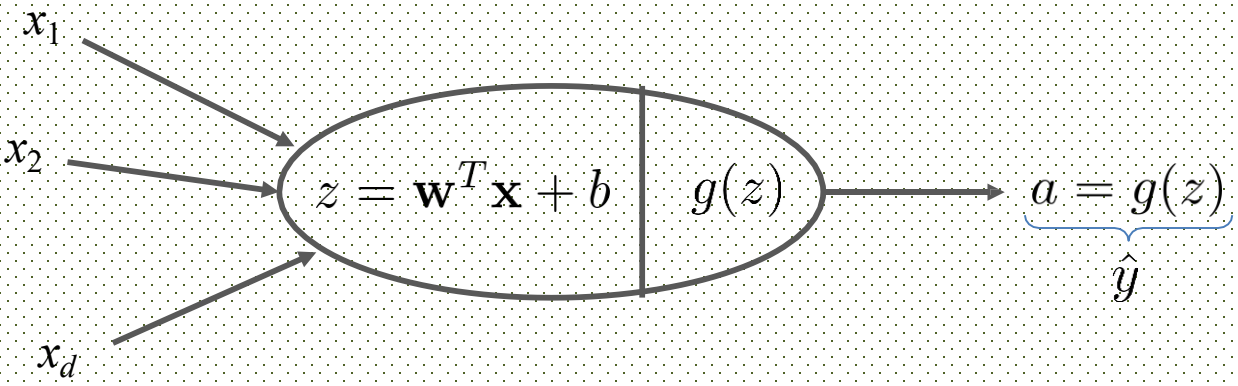


  - Interconexiones de las neuronas



Redes Neuronales

- Unidad neuronal
  - Representa una “neurona” dentro de la red (también: unidad logística)

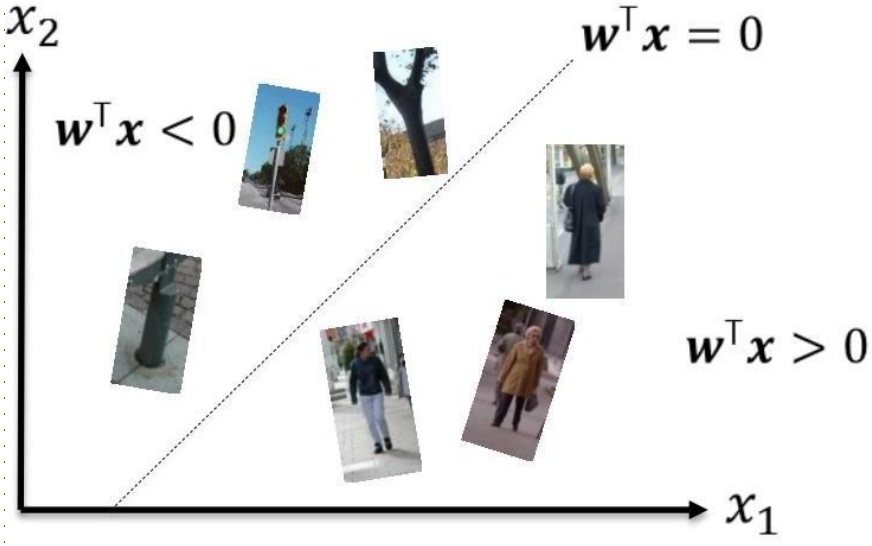


  - Partes importantes:
    - *Función de propagación* (combinación lineal de las entradas): $z \in \mathbb{R}$
      - Contiene pesos ($\boldsymbol{w} \in \mathbb{R}^d$) y bias ($b \in \mathbb{R}$)
    - *Función de activación*: $g \in \mathbb{R}$ y activación $a \in \mathbb{R}$

  - Entrada: $\boldsymbol{x} \in \mathbb{R}^d$ (d atributos)
  - Salida: $\hat{y} \in \mathbb{R}$

Redes Neuronales

- Unidad neuronal
  - La salida es una línea que divide el espacio de las características (según una frontera de decisión)



  - Para un nuevo objeto, se usa directamente los pesos entrenados para determinar su salida
  - Para curvas no lineales se usa varias neuronas (agrupadas en “capas”)

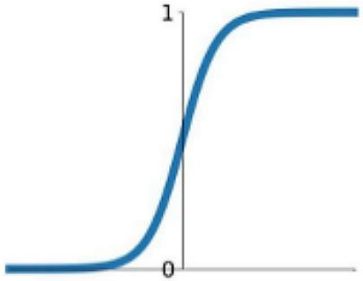
Redes Neuronales

Funciones de Activación

- Tienen como entrada la combinación lineal $z = w^T x + b$

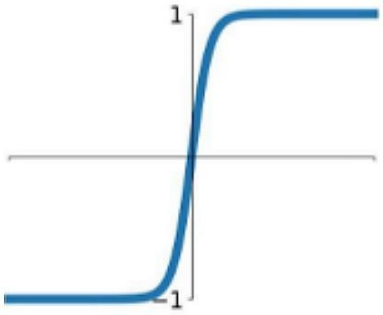
- Función sigmoidea
  - Llamada función logística
  - Mapea a (0,1)
  - Se usa para la salida de toda la red

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



- Función tangente hiperbólica
  - No tiene “bias” (centrado en 0)
  - Mapea a (-1, 1)
  - Se usa para capas ocultas

$$a(z) = \tanh(z)$$

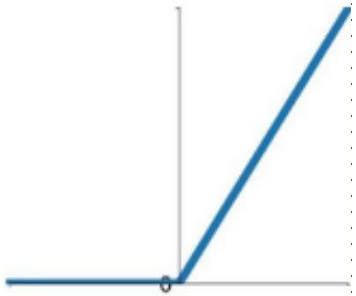


Redes Neuronales

Funciones de Activación

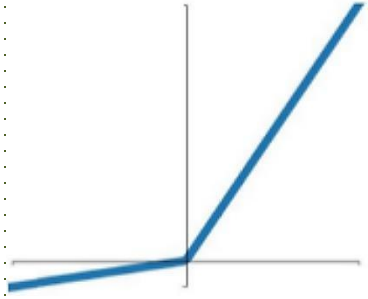
- Función ReLU
  - Significa: “Rectified Linear Unit”
  - Ventaja: facilidad de cálculo
  - Se suele usar en capas ocultas

$$a(z) = \text{máx}(0, z) = \begin{cases} 0, & \text{si } z < 0 \\ z, & \text{si } z \geq 0 \end{cases}$$



- Función Leaky ReLU
  - Variación de ReLU
  - El valor no se fija a cero: pequeña pendiente
  - Puede facilitar el “aprendizaje” (optimización)

$$a(z) = \text{máx}(0.01z, z) = \begin{cases} 0.01z, & \text{si } z < 0 \\ z, & \text{si } z \geq 0 \end{cases}$$



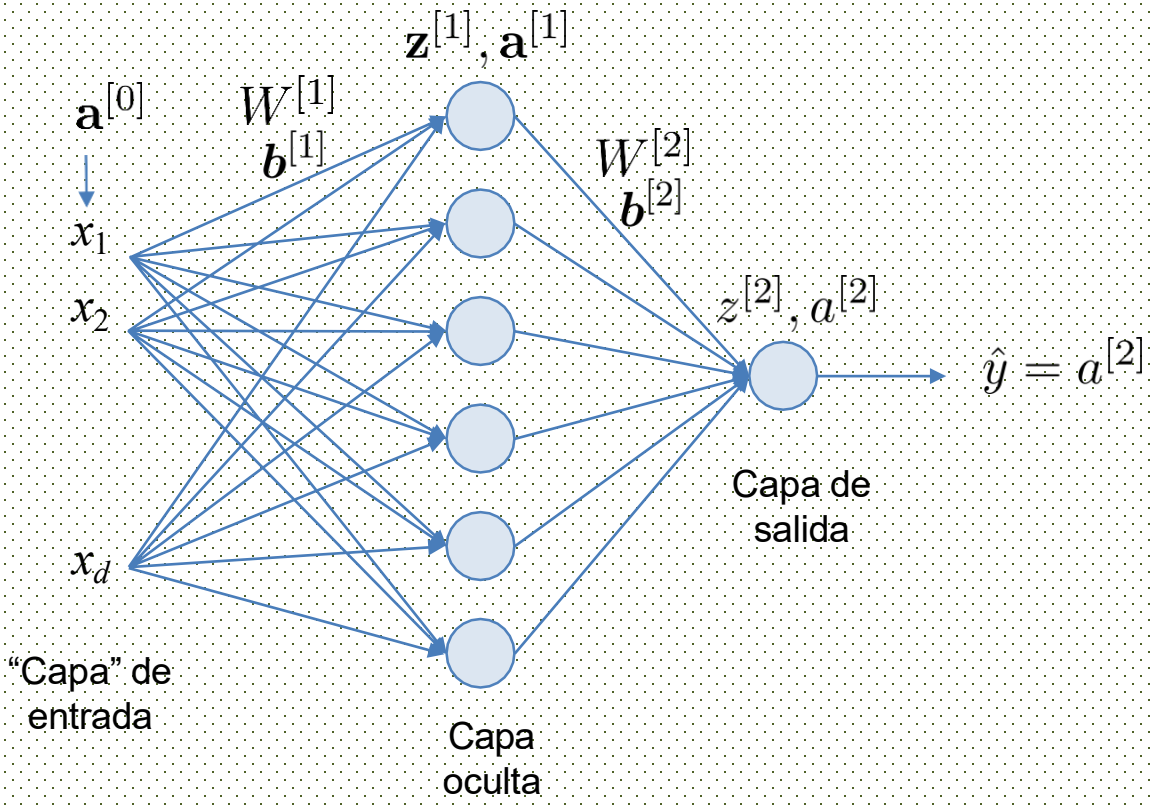
Temas

1. Introducción y Conceptos Básicos
2. **Redes Neuronales Simples**
3. Backpropagation
4. Redes Neuronales Profundas

Redes Neuronales Simples

Esquema con 2 capas

- Red neuronal con una capa oculta y una capa de salida (el corchete indica el número de capa)



- En general puede haber más de 1 salida (para clasificar varias clases)

Redes Neuronales Simples

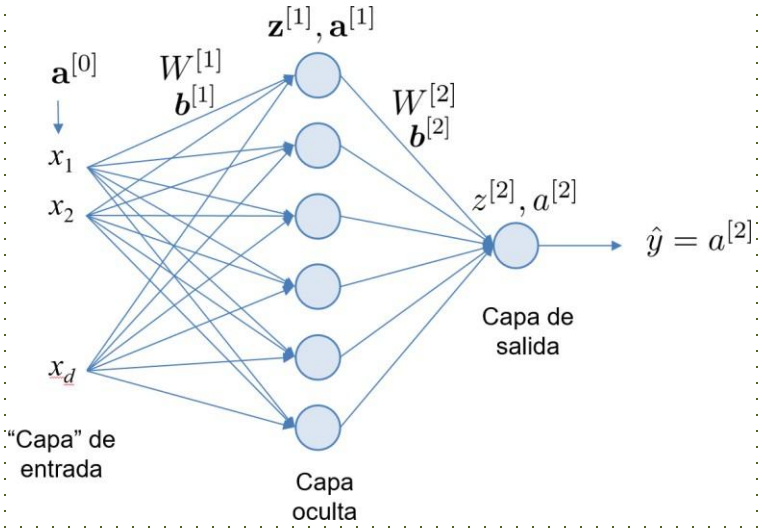
Esquema con 2 capas

- Elementos:

  - Entrada (capa de entrada): $\boldsymbol{x} = \boldsymbol{a}^{[0]} \in \mathbb{R}^d$

  - Capa oculta
    - Número de neuronas: $n^{[1]}$
    - Pesos $W^{[1]} \in \mathbb{R}^{n^{[1]} \times d}$, y bias $b^{[1]} \in \mathbb{R}^{n^{[1]}}$
    - Función de propagación: $\boldsymbol{z}^{[1]} \in \mathbb{R}^{n^{[1]}}$
    - Activación: $\boldsymbol{a}^{[1]} \in \mathbb{R}^{n^{[1]}}$

  - Capa de salida
    - Número de neuronas: $n^{[2]}$
    - Pesos $W^{[2]} \in \mathbb{R}^{1 \times n^{[1]}}$, y bias $b^{[2]} \in \mathbb{R}$
    - Función de propagación: $z^{[2]} \in \mathbb{R}$
    - Activación: $a^{[2]} \in \mathbb{R}$



Redes Neuronales Simples

Función de Hipótesis (2 capas)

- En la capa oculta: [1]

  - Funciones de propagación:

$$z_1^{[1]} = \boldsymbol{w}_1^{[1]T}\boldsymbol{x} + b_1^{[1]} \qquad z_2^{[1]} = \boldsymbol{w}_2^{[1]T}\boldsymbol{x} + b_2^{[1]} \qquad \cdots \qquad z_{n_1}^{[1]} = \boldsymbol{w}_{n_1}^{[1]T}\boldsymbol{x} + b_{n_1}^{[1]}$$

  - Funciones de activación

$$a_1^{[1]} = g(z_1^{[1]}) \qquad a_2^{[1]} = g(z_2^{[1]}) \qquad \cdots \qquad a_{n_1}^{[1]} = g(z_{n_1}^{[1]})$$

  - De forma vectorial:

$$\boldsymbol{z}^{[1]} = \begin{bmatrix} \boldsymbol{w}_1^{[1]T}\boldsymbol{x} + b_1^{[1]} \\ \boldsymbol{w}_2^{[1]T}\boldsymbol{x} + b_2^{[1]} \\ \vdots \\ \boldsymbol{w}_{n_1}^{[1]T}\boldsymbol{x} + b_{n_1}^{[1]} \end{bmatrix} \qquad \text{Si: } W^{[1]} = \begin{bmatrix} \boldsymbol{w}_1^{[1]T} \\ \boldsymbol{w}_2^{[1]T} \\ \vdots \\ \boldsymbol{w}_{n_1}^{[1]T} \end{bmatrix} \qquad \boldsymbol{b}^{[1]} = \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ \vdots \\ b_{n_1}^{[1]} \end{bmatrix}$$

$$\boldsymbol{z}^{[1]} = W^{[1]}\boldsymbol{x} + \boldsymbol{b}^{[1]} \qquad \boldsymbol{a}^{[1]} = g(\boldsymbol{z}^{[1]})$$

Redes Neuronales Simples

Función de Hipótesis (2 capas)

- En la capa de salida: [2]

  - Funciones de propagación: $z_i^{[2]} = \boldsymbol{w}_i^{[2]T} \boldsymbol{a}^{[1]} + b_i^{[2]}$

  - Funciones de activación: $a_i^{[2]} = g(z_i^{[2]})$

  - De forma vectorial:

$$W^{[2]} = \begin{bmatrix} \boldsymbol{w}_1^{[2]T} \\ \boldsymbol{w}_2^{[2]T} \\ \vdots \\ \boldsymbol{w}_{n_2}^{[2]T} \end{bmatrix} \quad \boldsymbol{b}^{[2]} = \begin{bmatrix} b_1^{[2]} \\ b_2^{[2]} \\ \vdots \\ b_{n_2}^{[2]} \end{bmatrix}$$

$$\boldsymbol{z}^{[2]} = W^{[2]} \boldsymbol{a}^{[1]} + \boldsymbol{b}^{[2]}$$
$$\boldsymbol{a}^{[2]} = g(\boldsymbol{z}^{[2]})$$

- Función de hipótesis (en la capa de salida):

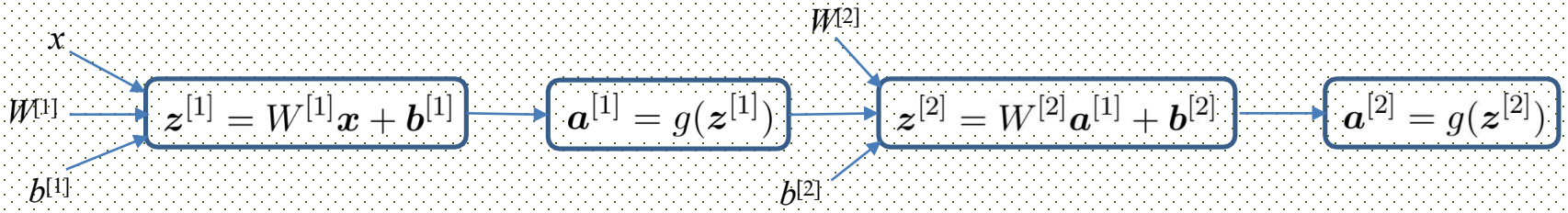
$$\boldsymbol{h}_w(\boldsymbol{x}) = \boldsymbol{a}^{[2]}$$

Asumiendo que hay 2 capas

Redes Neuronales Simples

Propagación hacia adelante

- Esquema con 2 capas



- Etapas:

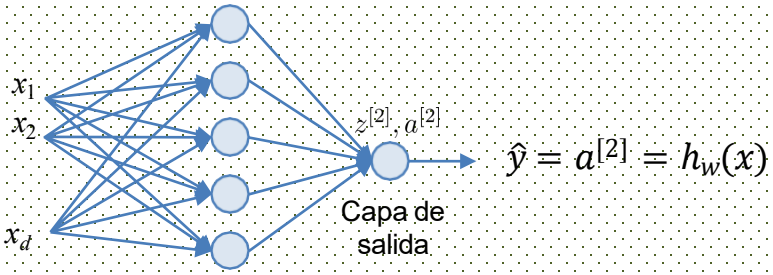
1. *Parámetros de la capa 1*
   - Entrada a la capa 1: instancias de entrenamiento $x = a^{[0]}$
   - Salida de la capa 1: activación $a^{[1]}$ de la capa 1
2. *Parámetros de la capa 2 a partir de la capa 1*
   - Entrada a la capa 2: activación $a^{[1]}$ de la capa 1
   - Salida de la capa 2: activación $a^{[2]}$ de la capa 2

- Los datos (y los cálculos) se propagan de “izquierda a derecha”: propagación hacia adelante.

Redes Neuronales Simples

Función de Costo

- Si solo hay 1 clase
  - Se usa una sola unidad neuronal (sigmoidea) a la salida



  - Función de costo:

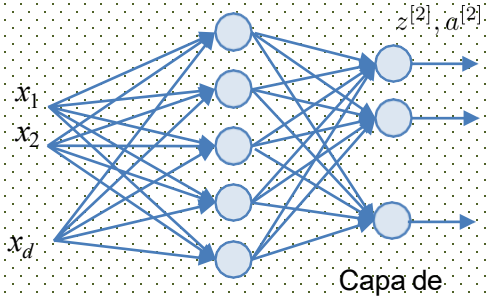
$$J(\boldsymbol{w}) = -\frac{1}{n}\sum_{i=1}^{n} y^{(i)} \log(h_w(\boldsymbol{x}^{(i)})) + (1 - y^{(i)}) \log(1 - h_w(\boldsymbol{x}^{(i)}))$$

  - n: número de instancias (de entrenamiento)
  - $y^{(i)}$: clase real de la muestra i (0 o 1)
  - $h_w(\boldsymbol{x}^{(i)})$: predicción para la muestra i (función de hipótesis)

Redes Neuronales Simples

Función de Costo

- Si hay K clases
  - Se usa K unidades neuronales a la salida (varias sigmoideas: "softmax")



$$\hat{y} = \boldsymbol{a}^{[2]} = \boldsymbol{h}_w(\boldsymbol{x}) = \begin{bmatrix} (h_w(x))_1 \\ (h_w(x))_2 \\ \vdots \\ (h_w(x))_K \end{bmatrix}$$

  - Función de costo

$$J(\boldsymbol{w}, \boldsymbol{b}) = -\frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} y_k^{(i)} \log(h_w(\boldsymbol{x}^{(i)}))_k + (1 - y_k^{(i)}) \log(1 - (h_w(\boldsymbol{x}^{(i)}))_k)$$

- Minimización
  - *Backpropagation*: calcula las derivadas de J con respecto a $\boldsymbol{w}_i$ y b_i
  - Se usa "gradient descent" para encontrar $\boldsymbol{w}_i, \boldsymbol{b}_i$ óptimos: $w_i = w_i - \alpha \frac{\partial J}{\partial w_i}$

Redes Neuronales Simples

Propagación

- Propagación hacia adelante ("predicción"): obtiene la predicción

$$\mathbf{z}^{[1]} = W^{[1]}\mathbf{a}^{[0]} + b^{[1]}$$
$$\mathbf{a}^{[1]} = g^{[1]}(\mathbf{z}^{[1]})$$

para la capa oculta (capa 1)

$$\mathbf{z}^{[2]} = W^{[2]}\mathbf{a}^{[1]} + b^{[2]}$$
$$\mathbf{a}^{[2]} = g^{[2]}(\mathbf{z}^{[2]}) = \hat{y}$$

para la capa de salida (capa 2)

- Propagación hacia atrás ("backpropagation"): obtiene las derivadas parciales

$$d\mathbf{z}^{[2]} = \mathbf{a}^{[2]} - \mathbf{y}$$
$$dW^{[2]} = d\mathbf{z}^{[2]}\mathbf{a}^{[1]T}$$
$$db^{[2]} = d\mathbf{z}^{[2]}$$

asumiendo que $a^{[2]}$ es sigmoidea

para la capa de salida (capa 2)

$$dz^{[1]} = W^{[2]T}d\mathbf{z}^{[2]} * \mathbf{g}^{[1]'}(\mathbf{z}^{[1]})$$
$$dW^{[1]} = d\mathbf{z}^{[1]}\mathbf{x}^{T}$$
$$db^{[1]} = d\mathbf{z}^{[1]}$$

multiplicación término a término

para la capa oculta (capa 1)

Notación

$$\frac{\partial J}{\partial \mathbf{z}} = d\mathbf{z}$$
$$\frac{\partial J}{\partial W} = dW$$
$$\frac{\partial J}{\partial b} = db$$
$$\mathbf{g}^{[1]'}(\mathbf{z}^{[1]}) = \frac{dg^{[1]}}{d\mathbf{z}^{[1]}}$$

Redes Neuronales Simples

Propagación usando varias Instancias a la vez

- El entrenamiento se realiza con n instancias
  - Todas las instancias $\boldsymbol{x}^{(i)} \in \mathbb{R}^d$ se agrupan en una matriz $X \in \mathbb{R}^{d \times n}$ para evitar bucles en la implementación

$$X = \begin{bmatrix} \boldsymbol{x}^{(1)} & \boldsymbol{x}^{(2)} & \cdots & \boldsymbol{x}^{(n)} \end{bmatrix}$$

  - Las propagaciones usan $A^{[0]} = X$:

    Se genera matrices $Z^{[1]} = [z^{[1](1)}, z^{[1](2)}, ..., z^{[1](n)}]$ y $A^{[1]} = [a^{[1](1)}, a^{[1](2)}, ..., a^{[1](n)}]$

- Propagación hacia adelante ("predicción")

$$\left.\begin{aligned} Z^{[1]} &= W^{[1]} A^{[0]} + \mathbf{b}^{[1]} \\ A^{[1]} &= g^{[1]}(Z^{[1]}) \end{aligned}\right\}$$ para la capa oculta (capa 1)

$$\left.\begin{aligned} Z^{[2]} &= W^{[2]} A^{[1]} + \mathbf{b}^{[2]} \\ A^{[2]} &= g^{[2]}(Z^{[2]}) \end{aligned}\right\}$$ para la capa de salida (capa 2)

Redes Neuronales Simples

Propagación usando varias Instancias a la vez

- Propagación hacia atrás ("backpropagation")

  - Para la capa de salida (asumiendo que la función de activación es sigmoidea)

$$\frac{\partial J}{\partial Z^{[2]}} = dZ^{[2]} = A^{[2]} - Y$$

$$\frac{\partial J}{\partial W^{[2]}} = dW^{[2]} = \frac{1}{n} dZ^{[2]} A^{[1]T}$$

$$\frac{\partial J}{\partial b^{[2]}} = db^{[2]} = \frac{1}{n} dZ^{[2]T} dZ^{[2]}$$

  - Para la capa oculta

$$\frac{\partial J}{\partial Z^{[1]}} = dZ^{[1]} = W^{[2]T} dZ^{[2]} * \mathbf{g}^{[1]'}(Z^{[1]})$$

$$\frac{\partial J}{\partial W^{[1]}} = dW^{[1]} = \frac{1}{n} dZ^{[1]} X^{T}$$

$$\frac{\partial J}{\partial b^{[1]}} = db^{[1]} = \frac{1}{n} dZ^{[1]T} dZ^{[1]}$$

- Nota: la derivada de g es: $\mathbf{g}^{[1]'}(Z^{[1]}) = \frac{dg^{[1]}}{dZ^{[1]}}$

Redes Neuronales Simples

Propagación usando varias Instancias a la vez

- Algoritmo
  1. Inicializar los pesos
  2. Aplicar propagación hacia adelante para calcular la predicción

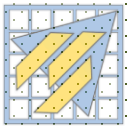
$$A^{[2]} = g^{[2]}(Z^{[2]})$$

  3. Aplicar propagación hacia atrás para calcular las derivadas de la función de costo (con respecto a w y b)
  4. Actualizar los pesos

$$w_i = w_i - \alpha \frac{\partial J}{\partial w_i} \qquad b = b - \alpha \frac{\partial J}{\partial b}$$

  5. Ir al paso 2, hasta satisfacer algún criterio de parada





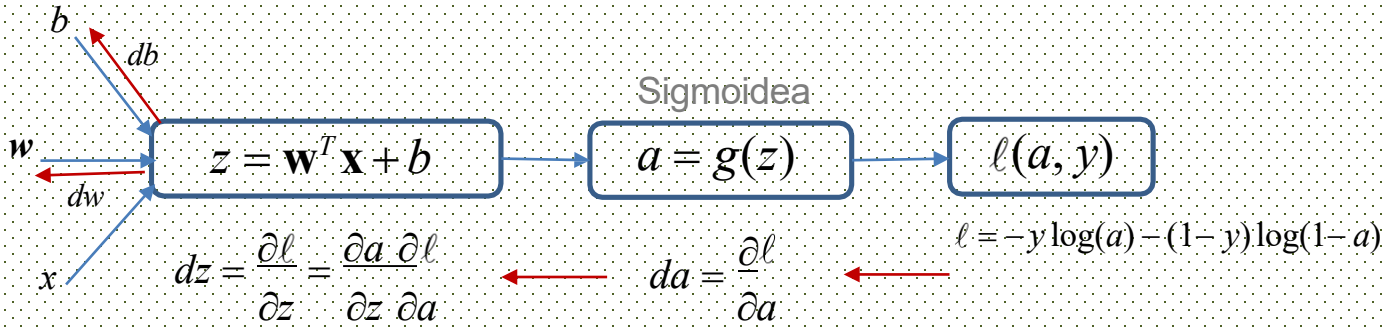
Temas

1. Introducción y Conceptos Básicos
2. Redes Neuronales Simples
3. **Backpropagation**
4. Redes Neuronales Profundas

Backpropagation

Cuando hay 1 sola Unidad Neuronal

- Esquema



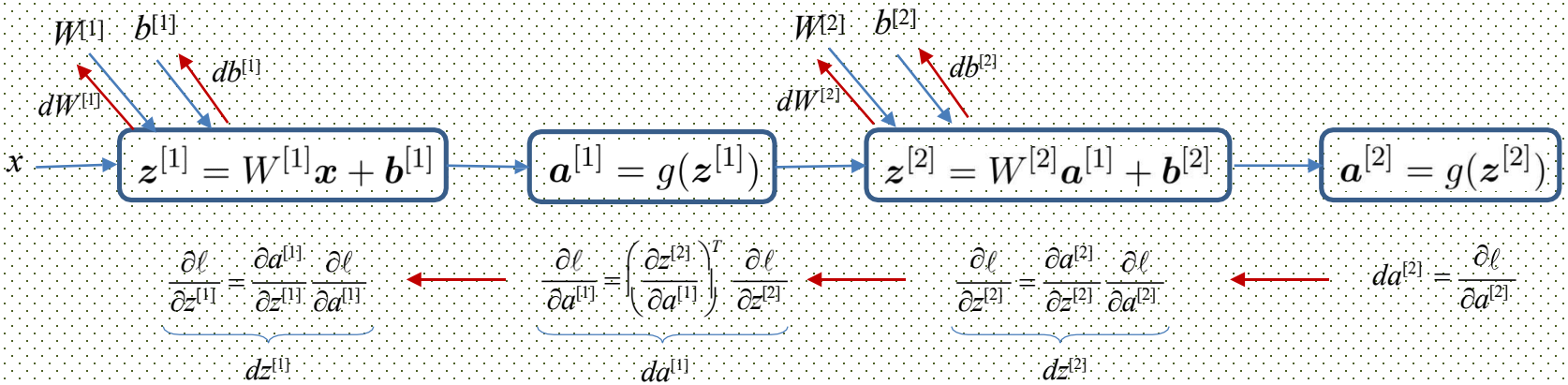
- Derivadas:

- da: $da = \frac{\partial \ell}{\partial a} = -\frac{y}{a} + \frac{1-y}{1-a}$
- dz: $\frac{\partial a}{\partial z} = a(1-a)$ ⇨ $dz = \frac{\partial \ell}{\partial z} = \frac{\partial a}{\partial z}\frac{\partial \ell}{\partial a} = a - y$ ⇨ $dz = a - y$
- dw: $dw = \frac{\partial \ell}{\partial w} = \frac{\partial z}{\partial w}\frac{\partial \ell}{\partial z}$ ⇨ $dw = \mathbf{x}\, dz$
- db: $db = \frac{\partial \ell}{\partial b} = \frac{\partial z}{\partial b}\frac{\partial \ell}{\partial z}$ ⇨ $db = dz$

Backpropagation

Cuando hay 2 Capas

- Esquema



Backpropagation

Cuando hay L capas

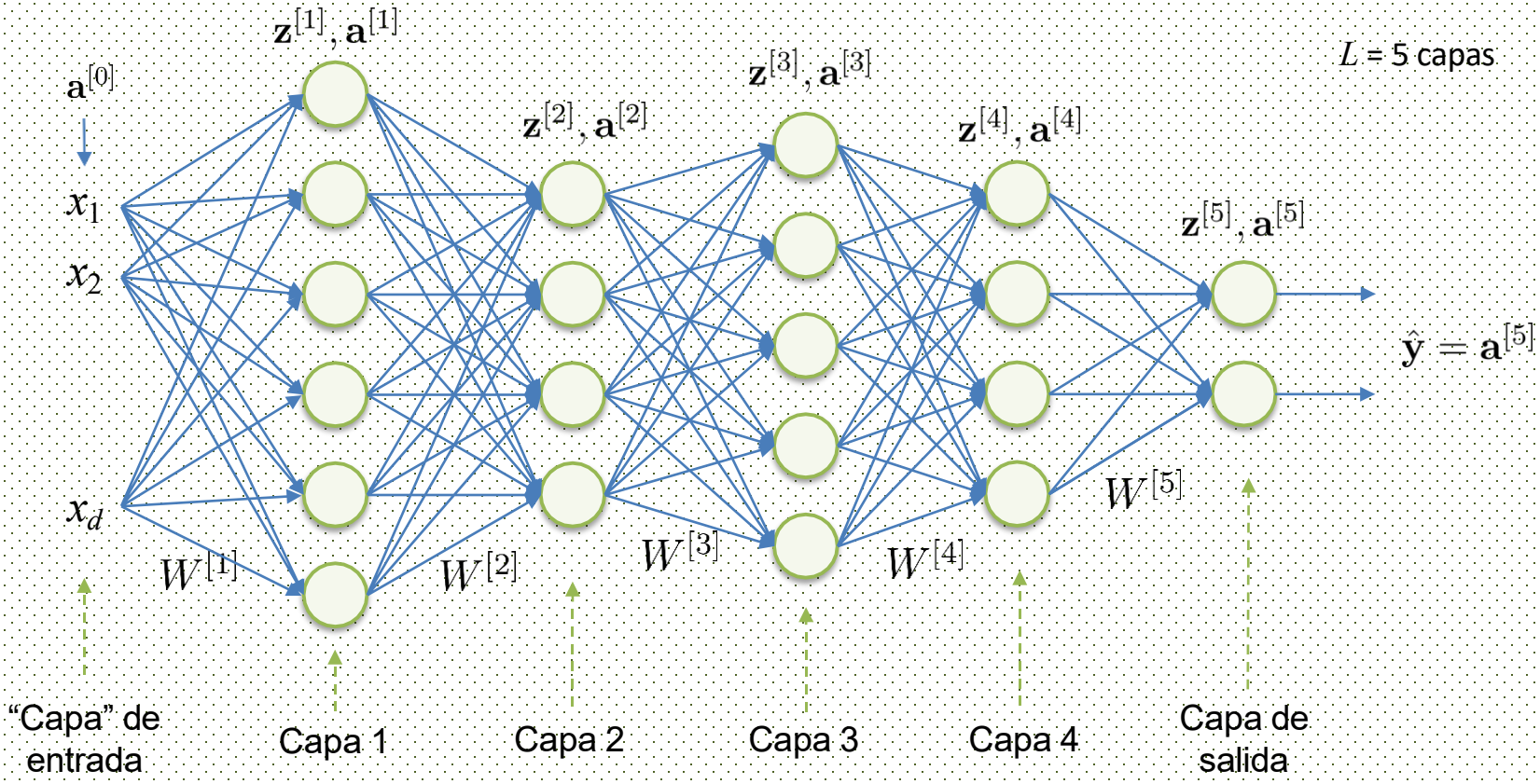
- Es una generalización de 2 capas
  - Se calcula las derivadas de derecha a izquierda
  - Se busca calcular $dW^{[l]}$, $db^{[l]}$

Temas

1. Introducción y Conceptos Básicos
2. Redes Neuronales Simples
3. Backpropagation
4. **Redes Neuronales Profundas**

Redes Neuronales Profundas

- Llamadas: *Deep Neural Networks* (*DNN*)
- Son redes neuronales con varias capas



Redes Neuronales Profundas

Elementos y Notación

- Elementos:

  - Entrada (capa de entrada): $\boldsymbol{x} = \boldsymbol{a}^{[0]} \in \mathbb{R}^{d}$

  - Capa oculta l (en total hay L-1 capas ocultas)
    - Número de neuronas en la capa l: $n^{[l]}$
    - Pesos $W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$, y bias $b^{[l]} \in \mathbb{R}^{n^{[l]}}$
    - Función de propagación: $\boldsymbol{z}^{[l]} \in \mathbb{R}^{n^{[l]}}$
    - Activación: $\boldsymbol{a}^{[l]} \in \mathbb{R}^{n^{[l]}}$

  - Capa de salida (capa L)
    - Número de neuronas: $n^{[L]}$
    - Pesos $W^{[L]} \in \mathbb{R}^{n^{[L]} \times n^{[L-1]}}$, y bias $b^{[L]} \in \mathbb{R}^{n^{[L]}}$
    - Función de propagación: $\boldsymbol{z}^{[L]} \in \mathbb{R}^{n^{[L]}}$
    - Activación (predicción): $\hat{y} = \boldsymbol{a}^{[L]} \in \mathbb{R}^{n^{[L]}}$

El número de neuronas de la capa de salida indica el número de clases que se desea clasificar

Redes Neuronales Profundas

Propagación hacia adelante

- En la capa l:

$$\mathbf{z}^{[l]} = W^{[l]}\mathbf{a}^{[l-1]} + b^{[l]}$$

$$\mathbf{a}^{[l]} = g^{[l]}(\mathbf{z}^{[l]})$$

- Se calcula los parámetros de la capa l a partir de la capa anterior (l - 1)
  - Entrada a capa l: activación $a^{[l-1]}$ de la capa anterior (l - 1)
  - Salida de capa l: activación $a^{[l]}$ de la capa actual (l)



- Los datos (y los cálculos) se propagan de “izquierda a derecha”: propagación hacia adelante.

Redes Neuronales Profundas

Función de Costo

- Con 1 sola clase de salida (1 unidad neuronal a la salida):

$$J(\boldsymbol{w}) = -\frac{1}{n}\sum_{i=1}^{n} y^{(i)} \log(h_w(\boldsymbol{x}^{(i)})) + (1 - y^{(i)}) \log(1 - h_w(\boldsymbol{x}^{(i)}))$$

  - n: número de instancias (de entrenamiento)
  - $y^{(i)}$: clase real de la muestra i (0 o 1)
  - $h_w(\boldsymbol{x}^{(i)})$: predicción para la muestra i (función de hipótesis)

- Con K clases a la salida

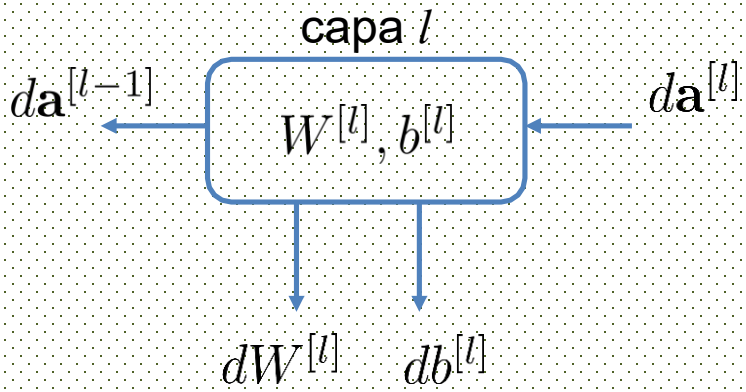
$$J(\boldsymbol{w}, \boldsymbol{b}) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_k^{(i)} \log(h_w(\boldsymbol{x}^{(i)}))_k + (1 - y_k^{(i)}) \log(1 - (h_w(\boldsymbol{x}^{(i)}))_k)$$

- Se minimiza la función de costo usando "*backpropagation*":
  - Obtiene los valores de las derivadas de J con respecto a $\boldsymbol{w}_i$ y b_i
  - Se usa "gradient descent" para encontrar $\boldsymbol{w}_i$, $\boldsymbol{b}_i$ óptimos: $w_i = w_i - \alpha \frac{\partial J}{\partial w_i}$

Redes Neuronales Profundas

Propagación hacia atrás

- En la capa l:



- Se calcula las derivadas de la capa l-1 a partir de la capa posterior (l)

$$dz^{[l]} = da^{[l]} * g^{[l]'}(z^{[l]})$$

$$dW^{[l]} = dz^{[l]} a^{[l-1]}$$

$$db^{[l]} = dz^{[l]}$$

$$da^{[l-1]} = W^{[l]T} dz^{[l]}$$

Redes Neuronales Profundas

Con *n* Instancias

- Entrada: $X = A^{[0]} \in \mathbb{R}^{d \times n}$
- Propagación hacia adelante (capa l)

$$Z^{[l]} = W^{[l]} A^{[l-1]} + \mathbf{b}^{[l]} \qquad Z^{[l]} = \begin{bmatrix} \mathbf{z}^{[l](1)} & \mathbf{z}^{[l](2)} & \dots & \mathbf{z}^{[l](n)} \end{bmatrix}$$

$$A^{[l]} = g^{[l]}(Z^{[l]}) \qquad A^{[l]} = \begin{bmatrix} \mathbf{a}^{[l](1)} & \mathbf{a}^{[l](2)} & \dots & \mathbf{a}^{[l](n)} \end{bmatrix}$$

- Propagación hacia atrás (capa l)

$$dZ^{[l]} = dA^{[l]} * \mathbf{g}^{[l]'}(Z^{[l]})$$

$$dW^{[l]} = \frac{1}{n} dZ^{[l]} A^{[l-1]T}$$

$$db^{[l]} = \frac{1}{n} dZ^{[l]T} dZ^{[l]}$$

En Python: `1/n*np.sum(dZ, axis=1, keepdims=True)`

$$dA^{[l-1]} = W^{[l]T} dZ^{[l]}$$

Redes Neuronales Profundas

Con n Instancias

- Inicialización
  - Los pesos $W^{[l]}$ deben inicializarse aleatoriamente con valores pequeños (si se inicializan con 0, las neuronas tendrían los mismos pesos: "simetría")
  - En la propagación hacia adelante, $A^{[0]}$ es X (la entrada)
  - En la propagación hacia atrás $dA^{[l]}$ usa la derivada directa

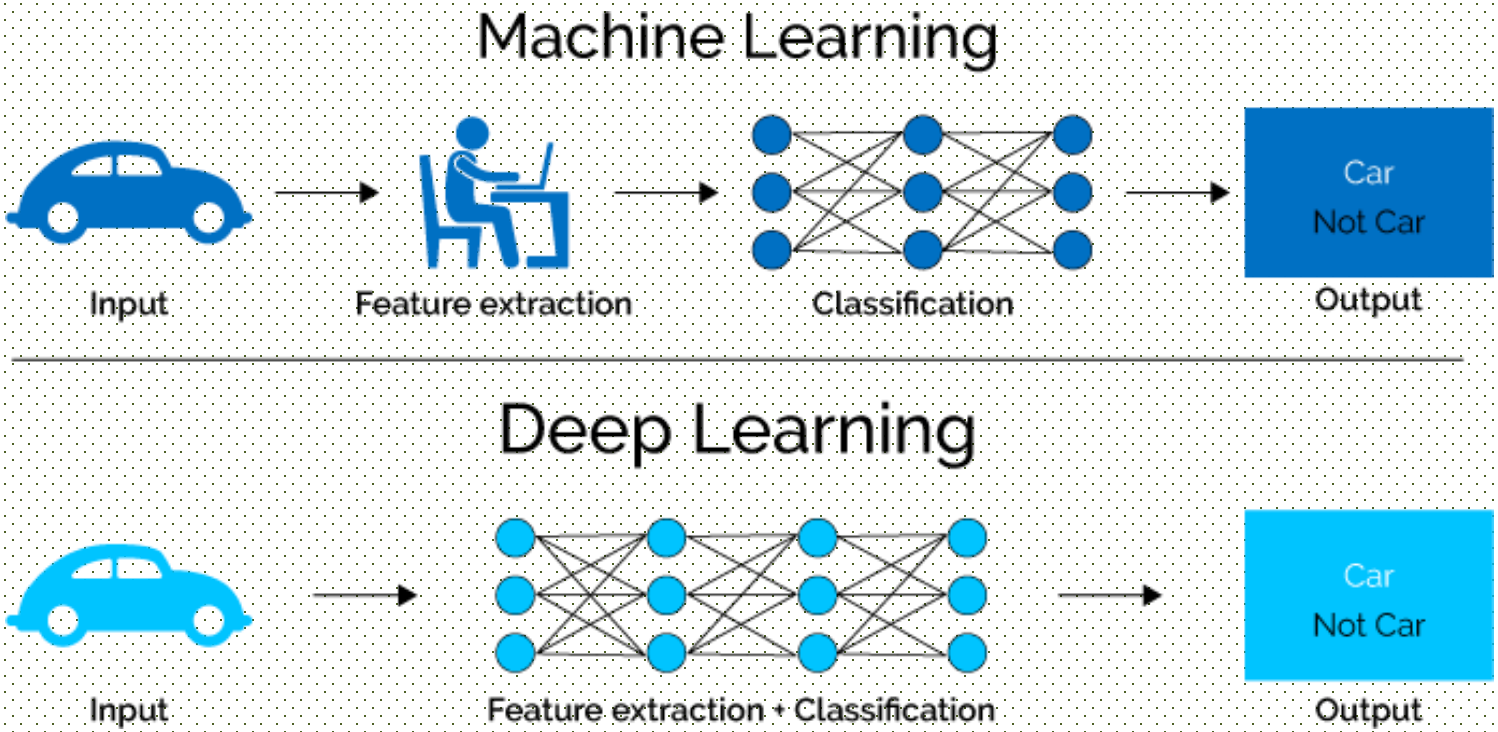
- En cada iteración
  - Propagación hacia adelante: de $l = 1$ (capa 1) hasta $l = L$ (última capa)
  - Cálculo de la función de costo J
  - Propagación hacia atrás: de $l = L$ hasta $l = 1$
    - Calcula $dW^{[l]}$, $db^{[l]}$ para cada capa
  - Actualización de los pesos en cada capa:

$$W[l] = W^{[l]} - \alpha dW^{[l]}$$

$$b[l] = b^{[l]} - \alpha db^{[l]}$$

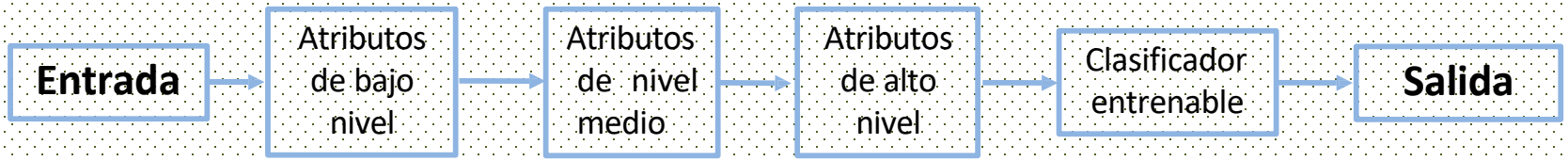
Introducción

- ¿Qué es Deep Learning?
  - Red neuronal con varias capas ocultas
  - Las capas ocultas realizan “identificación de características”
  - Solo funciona bien si hay muchos datos



Introducción

- En Deep Learning:

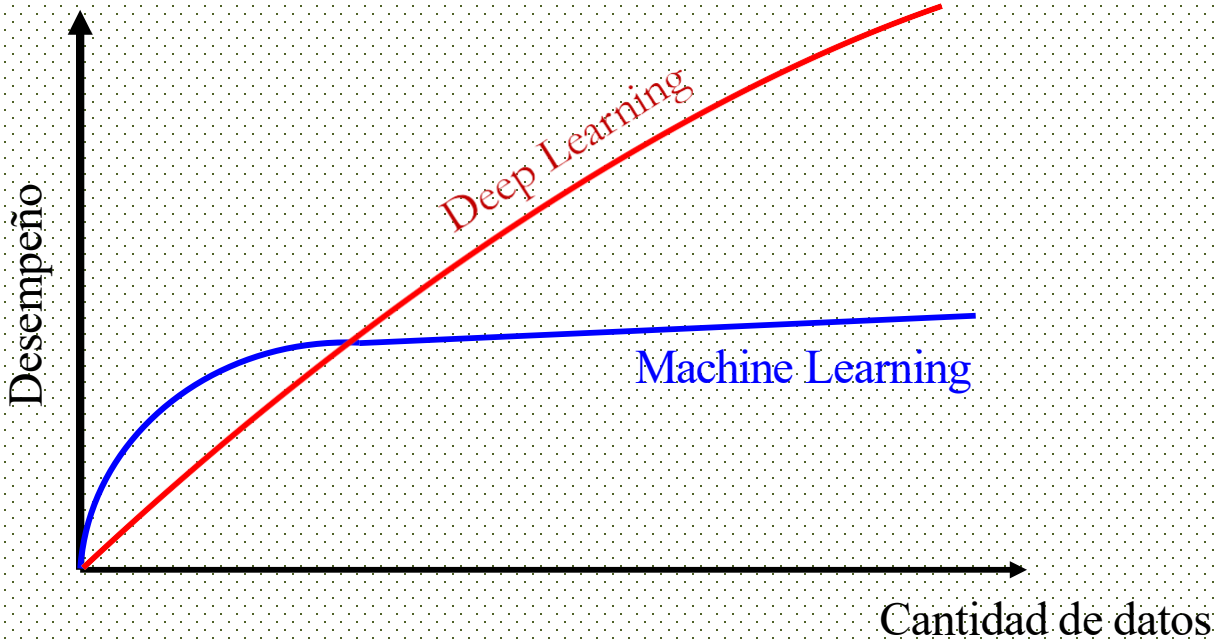


- *Cada parte de una red neuronal profunda transforma su representación en un nivel de abstracción más alto*

- *Ejemplo*
  - Imagen
    - Pixel → Borde → Textura → Parte → Objecto
  - Texto
    - Caracter → Palabra → Grupo de palabras → Proposición → Oración → Historia

Introducción

- Para poder aprender (y extraer características adecuadas), Deep Learning usa una gran cantidad de datos



- Gran cantidad de datos implica gran cantidad de parámetros
  - Se requiere mejores algoritmos de optimización
  - Es necesario realizar regularización

Regularización

Regularización de Norma

- Se utiliza alguna norma para regularizar los pesos
  - Usualmente: norma de Frobenius:

$$\left\|W^{[l]}\right\|_F^2 = \sum_{i=1}^{n^{[l-1]}} \sum_{j=1}^{n^{[l]}} (w_{ij}^{[l]})^2 \qquad W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$$

- Función de costo regularizada (asumiendo L capas y n instancias)

$$J = \frac{1}{n} \sum_{i=1}^{n} \ell(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2n} \sum_{l=1}^{L} \left\|W^{[l]}\right\|_F^2$$

- Derivada de la función de regularización

$$\frac{\partial}{\partial w_{ij}^{[l]}} \left(\frac{\lambda}{2n} \sum_{i=1}^{n^{[l-1]}} \sum_{j=1}^{n^{[l]}} (w_{ij}^{[l]})^2 \right) = \frac{\lambda}{n} w_{ij}^{[l]} \quad \Rightarrow \quad \frac{\partial}{\partial W^{[l]}} \left(\frac{\lambda}{2n} \sum_{l=1}^{L} \left\|W^{[l]}\right\|_F^2 \right) = \frac{\lambda}{n} W^{[l]}$$

  - Esta derivada solo afecta la actualización de los pesos (no del bias)

Regularización

Regularización de Norma

- Actualización de parámetros

$$W^{[l]} = W^{[l]} - \alpha \left(dW^{[l]} + \frac{\lambda}{2n} W^{[l]} \right)$$

$$b^{[l]} = b^{[l]} - \alpha db^{[l]}$$

- La actualización de los pesos se puede factorizar: “decaimiento ponderado”

$$W^{[l]} = W^{[l]} - \alpha \frac{\lambda}{2n} W^{[l]} - \alpha dW^{[l]}$$

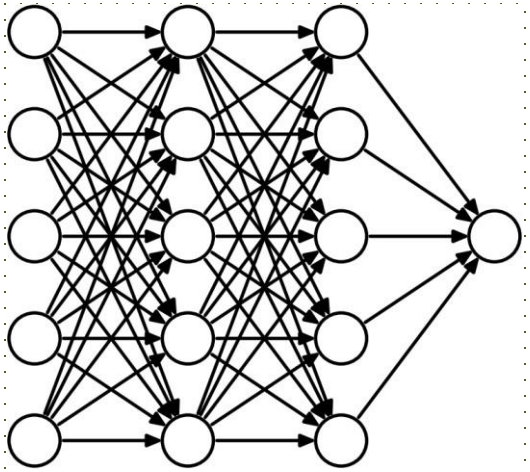
$$W^{[l]} = \underbrace{\left(1 - \alpha \frac{\lambda}{2n} \right)}_{\text{menor a 1}} W^{[l]} - \alpha dW^{[l]}$$

Regularización

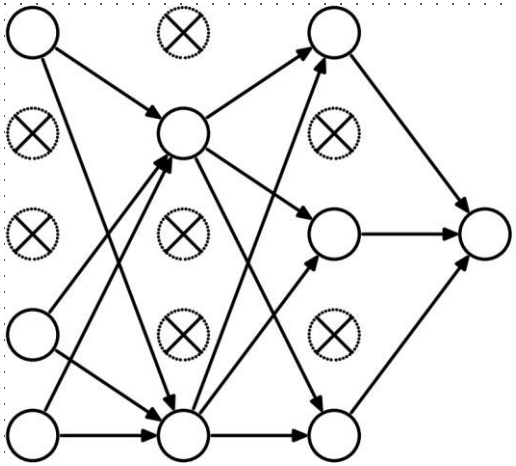
Dropout

- Retira un nodo de cada capa de manera aleatoria
  - Durante entrenamiento: cada neurona tiene probabilidad p de estar presente
  - Durante prueba: todas las neuronas están presentes

- Ejemplo: durante el entrenamiento



Red neuronal inicial

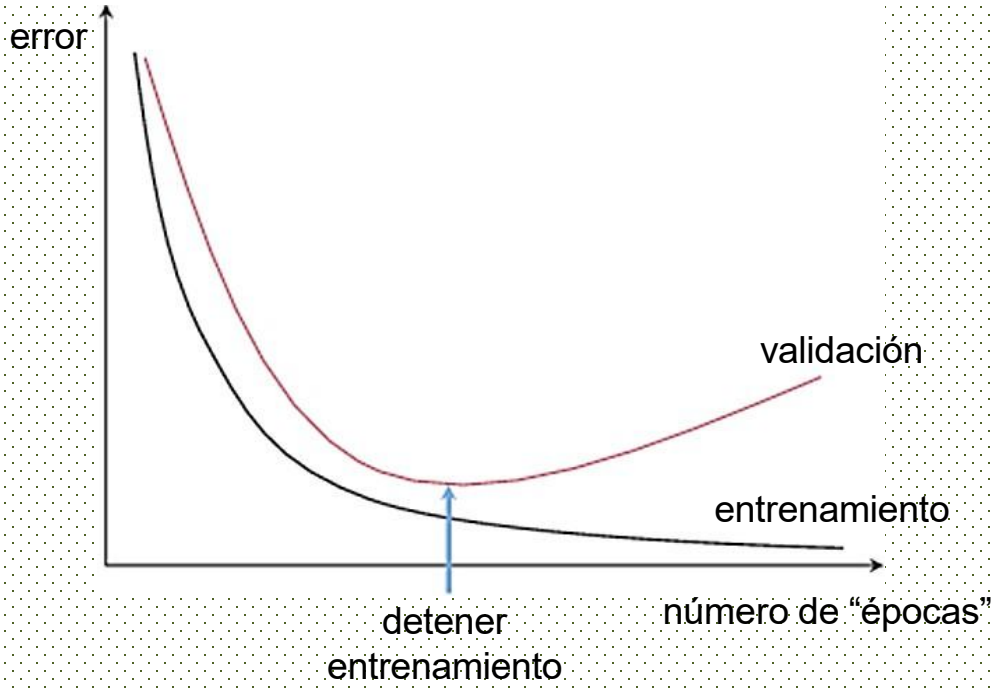


Red neuronal con “dropout”
(las X indican que no se usan)

Regularización

"Early Stopping"

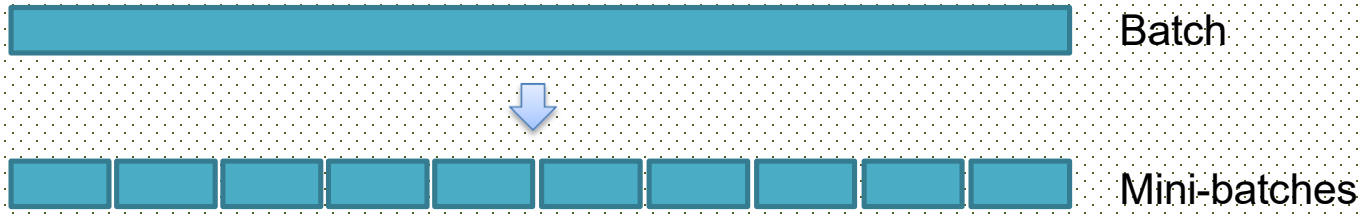
- Se analiza el error en el conjunto de entrenamiento y en conjunto de validación
- Se detiene el algoritmo antes de que realice "overfitting"



Algoritmos de Optimización

Descenso del Gradiente Mini-Batch

- Se divide todos los datos de entrenamiento (“batch”) en partes (“mini-batches”)



- El bucle utiliza cada uno de los mini-batches
  - En cada actualización de parámetros se utiliza 1 mini-batch (diferente)
  - La derivada que se usa es “aproximada” (no usa todo el batch)
  - Reduce el tiempo de entrenamiento

- Época (“epoch”): una “pasada” completa a todos los datos (todos los mini-batches)

Algoritmos de Optimización

Métodos Principales

- Son modificaciones del descenso del gradiente
- Métodos:
  - Descenso de gradiente con momentum
  - Gradiente acelerado de Nesterov
  - Adagrad
  - RMSprop
  - Algoritmo de Adam
  - AdaMax
  - Nadam
  - AMSGrad
  - AdamW
  - QHAdam
  - AggMo
  - YellowFin
  - QMH

Referencias

- Bishop, C. (2006) *Pattern recognition and machine learning*. Springer

- Hastie, T., Tibshirani, R., Friedman, J. (2009). *The elements of statistical learning*. Springer Science & Business Media.

- Murphy, K. (2012). *Machine Learning, A Probabilistic Perspective*. The MIT Press.